

Case Técnico: Modernização do Portal Pessoas

Contexto

O **Portal Pessoas** é o portal corporativo utilizado por milhares de colaboradores para acessar jornadas, serviços e informações relacionadas ao dia a dia de trabalho.

Atualmente:

- Possui experiência **web** e **mobile**.
- Parte das jornadas ainda está em plataformas legadas.
- Existem diversas squads desenvolvendo funcionalidades simultaneamente.
- A empresa deseja evoluir para uma arquitetura moderna, escalável e flexível.
- O app deverá possuir uma camada nativa mínima, priorizando reuso de código entre web e mobile.
- A solução precisa permitir que novos times sejam adicionados sem gerar acoplamento excessivo.
- A experiência do colaborador deve ser consistente entre web e mobile.
- A arquitetura deve permitir evolução incremental, convivendo com o legado durante o período de migração.

Desafio

Você foi contratado como **Frontend Senior** para propor a arquitetura da próxima geração do Portal Pessoas.

Sua missão é apresentar uma solução técnica e desenvolver uma aplicação pequena que represente, de forma simplificada, os principais desafios do cenário real.

A aplicação não precisa estar publicada nem pronta para produção, mas deve demonstrar de forma clara as principais decisões arquiteturais da solução proposta.

Objetivo do Case

Avaliar sua capacidade de:

- Propor uma arquitetura frontend escalável.
- Pensar em uma solução compatível com times trabalhando em paralelo.
- Definir uma estratégia de reuso entre web e mobile.
- Criar uma estrutura flexível, preparada para evolução.
- Representar em código uma solução simples, mas coerente com a complexidade do cenário.
- Justificar trade-offs técnicos de forma clara.

Requisitos Funcionais

A solução proposta deve considerar:

- Home personalizada.
- Catálogo de jornadas.

- Busca global.
- Notificações.
- Navegação consistente entre web e mobile.
- Possibilidade de convivência entre jornadas modernas e jornadas legadas.
- Inclusão de novas jornadas sem necessidade de alterar o core da aplicação.

Requisitos Não Funcionais

Escalabilidade Organizacional

A arquitetura deve suportar:

- 10+ squads trabalhando simultaneamente.
- Separação por domínios ou jornadas.
- Deploy independente por domínio, quando aplicável.
- Baixa dependência entre equipes.
- Governança mínima para evitar conflitos entre squads.

Reuso

A solução deve considerar:

- Máximo compartilhamento possível entre web e mobile.
- Design System único.
- Componentes reutilizáveis.
- Separação entre componentes de negócio e componentes visuais.
- Estratégia para evitar duplicação excessiva de código.

Flexibilidade e Evolução

A arquitetura deve permitir:

- Inclusão de novos produtos ou jornadas sem alterar o core.
- Substituição futura de tecnologias com impacto controlado.
- Evolução incremental, sem necessidade de big bang.
- Convivência temporária com plataformas legadas.
- Rollout gradual de novas funcionalidades.

Observabilidade

A solução deve prever:

- Logs.
- Métricas.
- Monitoramento de erros.
- Analytics de navegação.
- Rastreabilidade mínima para entender falhas em jornadas ou módulos.

Entregáveis Esperados

A avaliação será composta por duas partes complementares, podendo ser apresentada no formato que melhor comunique sua proposta:

1. **Proposta técnica da arquitetura**, descrevendo a solução, suas decisões técnicas e respectivos trade-offs.
2. **Implementação prática**, demonstrando os principais conceitos arquiteturais da solução proposta por meio de uma aplicação simplificada.

Parte 1: Proposta Técnica

1. Arquitetura de Alto Nível

Apresentar um desenho ou diagrama contendo, no mínimo:

- Shell Application.
- Microfrontends ou alternativa arquitetural escolhida.
- Mobile App.
- BFF.
- APIs.
- Design System.
- Observabilidade.
- Estratégia de integração com jornadas legadas.

O desenho pode ser feito em qualquer ferramenta ou até em formato textual, desde que seja claro.

2. Estratégia Web + Mobile

Explicar:

- O que fica na camada nativa do app.
- O que pode ser compartilhado entre web e mobile.
- Como reduzir duplicação de código.
- Como garantir consistência visual e funcional.
- Qual tecnologia ou abordagem seria utilizada.

Exemplos de possibilidades:

- WebView.
- PWA.
- Aplicação nativa mínima com experiências compartilhadas.
- Outra abordagem proposta pelo candidato.

A escolha deve ser justificada com seus respectivos trade-offs.

3. Estratégia para Múltiplas Squads

Explicar como a arquitetura permite que várias equipes trabalhem em paralelo.

A proposta deve abordar:

- Como as equipes publicam novas funcionalidades.
- Como evitar conflitos entre squads.
- Como funciona versionamento.
- Como funciona governança técnica.
- Como garantir padrões mínimos de qualidade.
- Como evitar que o shell vire um ponto central de dependência para todos os times.

4. Estratégia de Microfrontends ou Alternativa

Explicar se utilizaria ou não microfrontends.

Caso utilize, justificar:

- Module Federation.
- Web Components.
- Single SPA.
- Packages versionados.
- Monorepo.
- Outra abordagem.

Caso não utilize, explicar qual alternativa atenderia melhor ao cenário.

A resposta deve deixar claros os trade-offs envolvendo:

- Autonomia dos times.
- Complexidade operacional.
- Performance.
- Governança.
- Reuso.
- Evolução futura.

5. Design System

Explicar como seria estruturado o Design System.

A proposta deve considerar:

- Tokens.
- Componentes base.
- Componentes compostos.
- Componentes específicos de negócio.
- Versionamento.
- Governança.
- Estratégia para adoção por múltiplas squads.
- Compatibilidade entre web e mobile, quando aplicável.

6. Estratégia de Migração

Considerando que existem jornadas legadas, explicar:

- Como conviver com o legado.
- Como migrar sem big bang.
- Como redirecionar o usuário para jornadas ainda não modernizadas.
- Como controlar rollout.
- Como medir avanço da migração.
- Como evitar impacto na experiência do colaborador.

Parte 2: Aplicação Prática

Além da proposta técnica, o candidato deverá desenvolver uma aplicação pequena que represente a arquitetura proposta.

A aplicação não precisa estar publicada, mas deve ser entregue com instruções para execução local.

Objetivo da Aplicação

A aplicação deve simular, de forma simplificada, um portal corporativo com múltiplas jornadas e arquitetura preparada para evolução.

Ela não precisa ter todas as funcionalidades completas, mas deve demonstrar claramente como a arquitetura funcionaria em um cenário real com múltiplas squads, reuso e convivência com legado.

Requisitos Mínimos da Aplicação

A aplicação deve conter:

1. Shell ou Aplicação Principal

Deve existir uma aplicação principal responsável por:

- Estrutura geral da navegação.
- Carregamento das jornadas.
- Menu ou catálogo de funcionalidades.
- Layout base.
- Integração com componentes compartilhados.

2. Pelo menos 2 Jornadas Modernas

Criar pelo menos duas jornadas simulando domínios diferentes.

Exemplos:

- Registro de ponto.
- Benefícios.
- Férias.

- Holerite.
- Dados cadastrais.
- Solicitações internas.

Cada jornada deve estar organizada como se pudesse ser mantida por squads diferentes.

Orientação Final ao Candidato

A expectativa não é construir uma aplicação completa de produção, mas sim demonstrar capacidade de estruturar uma solução coerente para um cenário complexo.

A entrega deve mostrar:

- Visão arquitetural.
- Capacidade de simplificar um problema complexo.
- Clareza nas decisões.
- Código organizado.
- Preocupação com evolução.
- Preocupação com operação.
- Pensamento orientado a produto, experiência e escalabilidade.

Mais importante do que escolher a tecnologia “certa” é demonstrar domínio sobre os trade-offs e explicar como a solução atenderia um portal corporativo crítico, com múltiplas jornadas, múltiplas squads, necessidade de reuso e evolução incremental.